```
function [param,acheck_array,model_info,varfac_best,ens_coh] =
ps_ils_and_bootstrap_estimation(h2ph,Btemp,Bdop,dphase,model,std_param,sig2_est,
breakpoint,breakpoint2)

% Function for the estimation of the ambiguities and parameters of
% interest using integer least-squares (LAMBDA). Various
% alternative hypothesis models can be evaluated, untill a solution
% is found which leads to an accepted Overall Model Test.
%
% Input:  - h2ph                height to phase factors per arc
%         - Btemp               temporal baselines [year]
%         - Bdop                Doppler baselines [Hz]
%         - dphase              phase differences per arc
%         - model               vector with model indices to be
%                               evaluated (see help model_definitions
%                               for more information)
%         - std_param           standard deviations for pseudo-
%                               observations
%         - sig2_est            estimated variance components
%         - breakpoint          breakpoint in case of double
%                               linear model
%         - breakpoint2         second breakpoint, should be larger
%                               than first breakpoint
% Output: - param               estimated parameters
%         - acheck_array        array with estimated ambiguities
%         - model_info          array with model information
%         - varfac_best         a-posteriori variance factors
%         - ens_coh             ensemble coherence
%
%
% ------------------------------------------------------------------------
% File.............: ps_ils_and_bootstrap_estimation.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%



% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global althyp_index althyp_index2 Npar_max ps_method varfac_threshold

[Npoints,Nifgs] = size(dphase);
Nmodels = length(model);
model_index = 1;
if ~isempty(breakpoint)
    althyp_index = breakpoint;
else
    althyp_index = 2; % for proper flow of models
end
```

```
%%%sami
if ~isempty(breakpoint2)
    althyp_index2 = breakpoint2;
else
    althyp_index2 = NaN;
end
%%%sami

model_info = repmat(NaN,Npoints,2);

param = repmat(NaN,Npoints,Npar_max);
acheck_array = repmat(NaN,Nifgs,Npoints);
ens_coh = repmat(NaN,Npoints,1);
varfac_best = repmat(inf,Npoints,1);

mean_h2ph = mean(h2ph,1)';

if isempty(sig2_est)
    if model(1) == 1 % master atmosphere stochastic
        sig2 = [(pi*15/180)^2; repmat((pi*20/180)^2,Nifgs,1)];
    else % master atmosphere estimated
        sig2 = [0; repmat((pi*30/180)^2,Nifgs,1)];
    end
else
    sig2 = sig2_est;
end

Qy1 = repmat(2*sig2(1),Nifgs,Nifgs);
for v = 1:Nifgs
    Qy1(v,v) = Qy1(v,v)+2*sig2(v+1);
end


% --------------------------------------------------------------------
% Main loop to test alternative hypothesis
% --------------------------------------------------------------------

points_todo= 1:Npoints;

while ~isempty(points_todo) && (model_index<=Nmodels)

  fprintf(1,'Evaluating model %g of %g\n',model_index,Nmodels);
  fprintf(1,'Model: %g, alternative hypothesis: %g, alternative hypothesis2:
%g\n',model(model_index),althyp_index,althyp_index2);
  fprintf(1,'Points remaining: %g of %g\n\n',length(points_todo),Npoints);

  % ------------------------------------------------------------------
  % Construct mathematical model
  % ------------------------------------------------------------------

  [Npar,par_index] = ps_model_definitions('Npar',model(model_index));
  [B1Qy2,par_index] =
ps_model_definitions('model',model(model_index),Nifgs,mean_h2ph,Btemp,Bdop,std_p
aram);
  B1 = B1Qy2(1:Nifgs,:);
  B2 = eye(Npar);
  B = [B1;B2];

  A1 = -2*pi*eye(Nifgs);
  A2 = zeros(Npar,Nifgs);
  A = [A1;A2];

  C = [A B];
```

```
% --------------------------------------------------------------------
% Construct stochastic model
% --------------------------------------------------------------------

Qy2 = B1Qy2(Nifgs+1:end,:);
Qy12 = zeros(Nifgs,Npar);
Qy = [Qy1 Qy12;Qy12' Qy2];
invQy = inv(Qy);
invQy1 = inv(Qy1);


% --------------------------------------------------------------------
% Calculate covariance matrices
% --------------------------------------------------------------------

Qchat = inv(C'*invQy*C);
Qahat = Qchat(1:Nifgs,1:Nifgs);
%Qbhat = Qchat(Nifgs+1:end,Nifgs+1:end);
%Qabhat = Qchat(1:Nifgs,Nifgs+1:end);
%Qbahat = Qchat(Nifgs+1:end,1:Nifgs);

%Qbcond = Qbhat - Qbahat*inv(Qahat)*Qabhat;
%Qyhat = B1*Qbcond*B1';
%Qehat =Qy1-Qyhat;

Qbhat2 = inv(B1'*invQy1*B1);
rhs_fixed = Qbhat2*B1'*invQy1;
%Qbhat3 = inv(B1'*B1)
%cor2 = repmat(NaN,Npar,Npar);
%cor3 = repmat(NaN,Npar,Npar);
%for v = 1:Npar
%    for w = 1:Npar
%        cor2(v,w) = Qbhat2(v,w)/(sqrt(Qbhat2(v,v))*sqrt(Qbhat2(w,w)));
%        cor3(v,w) = Qbhat3(v,w)/(sqrt(Qbhat3(v,v))*sqrt(Qbhat3(w,w)));
%    end
%end
%cor2
%cor3

clear A A1 A2 B B2 B1Qy2 Qy Qy2 Qy12 invQy Qchat Qbhat2


% --------------------------------------------------------------------
% Calculate testing parameters
% --------------------------------------------------------------------

b = Nifgs-Npar; % redundancy


% --------------------------------------------------------------------
% Decorrelate Qahat
% --------------------------------------------------------------------

[Z,L,D] = decorrel_freek(Qahat);
invZ = inv(Z);
invL = inv(L);
invD = 1./D;

% --------------------------------------------------------------------
% Calculate the successrate
% --------------------------------------------------------------------

%srate = prod(erf(1./(2*sqrt(2*D))));
```

```
% ------------------------------------------------------------------------
% Loop to evaluate all arcs
% ------------------------------------------------------------------------

ncands = 1;
for v = points_todo

  %dummy = [v length(points_todo)];
  %if (rem(v,1000)==0)
  %  fprintf(1,'Processing point %5.0f of %5.0f\n',dummy);
  %end



  % ------------------------------------------------------------------------
  % Integer estimation
  % ------------------------------------------------------------------------

  y = dphase(v,:)';
  afloat = y/(-2*pi);
  zfloat = Z'*afloat;
  [Chi2,zcheck_boot] = ps_chistart(D,L,zfloat,ncands);

  switch ps_method
    case 'ils'
      %%    [zcheck,sqnorm,ierr] = bk_lsearch(zfloat,invL,invD,Chi2,ncands);
      [zcheck,sqnorm,ierr,backtrack_count,break_flag] = ...
          bk_lsearch_count(zfloat,invL,invD,Chi2,ncands);
      if break_flag == 1
          zcheck = zcheck_boot;
      end

    case 'boot'
      zcheck = zcheck_boot;

    otherwise
      error('Something went wrong with the specification of the ps_method');
  end

  acheck = round(zcheck' * invZ)';

  y_unwrap = y+2*pi*acheck;
  bcheck = rhs_fixed*y_unwrap;

  ycheck = B1*bcheck;
  echeck = y_unwrap - ycheck;



  % ------------------------------------------------------------------------
  % A-posteriori variance factor
  % ------------------------------------------------------------------------

  OMT = echeck'*invQy1*echeck; % Overall Model Test
  varfac = OMT/b; %a-posteriori variance factor

  if varfac<varfac_best(v)
    acheck_array(:,v) = acheck;
    model_info(v,:) = [model(model_index) althyp_index-1];
    param(v,par_index) = bcheck';
    ens_coh(v) = abs((1/Nifgs)*sum(exp(i*echeck)));
```

```
      varfac_best(v) = varfac;
    end

  end %for

  if (althyp_index == Nifgs)||(isnan(althyp_index))||~isempty(breakpoint)
    varfac_index = varfac_best(points_todo)>1; %try other model if >1
    points_todo = points_todo(varfac_index);
    model_index = model_index + 1;
    althyp_index = 2; % to start with 2
  end

end %while

acheck_array = acheck_array';

% ------------------------------------------------------------------------
% Remove wrong estimates
% ------------------------------------------------------------------------

%varfac_index = varfac_best>varfac_threshold; %keep arcs with reasonable results
%varfac_best(varfac_index) = NaN;
%param(varfac_index,:) = NaN;
%acheck_array(varfac_index,:) = NaN;
%model_info(varfac_index,:) = NaN;
%ens_coh(varfac_index) = NaN;
```